Alexandre Schneider 6844 FDU MSACS

GitHub Repo: GitHub - alxschneider/DistributedDataAccessLab

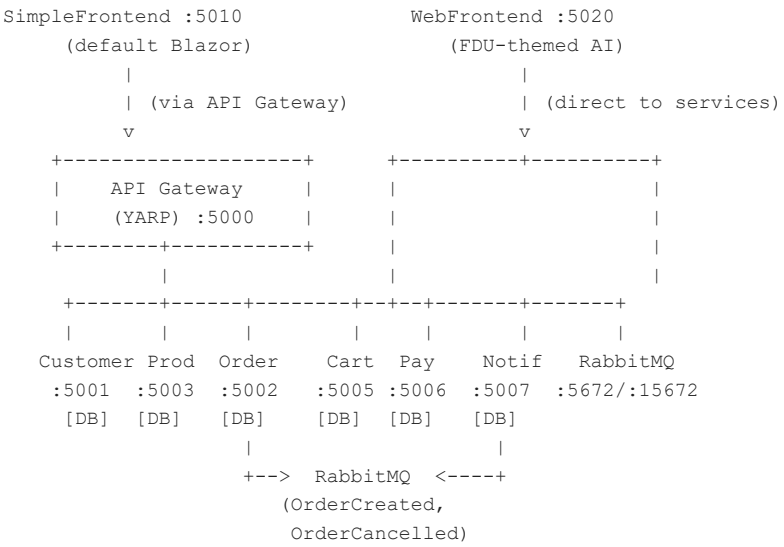
This report documents the third deliverable (V3) of the DistributedDataAccessLab project. It adds two Blazor Server frontends to the existing microservices backend: a SimpleFrontend and a WebFrontend. Both run as Docker containers alongside the 6 backend services, RabbitMQ, and the API Gateway, bringing the total to 10 containers managed by docker-compose.

Summary of Additions from V2

- SimpleFrontend (:5010) — minimal Blazor Server dashboard consuming all backend endpoints via API Gateway
- WebFrontend (:5020) — AI-generated FDU-themed e-commerce marketplace with full CRUD and role-based UI
- Docker Compose updated to 10 containers (6 services + RabbitMQ + API Gateway + 2 frontends)
- Two different API communication patterns: gateway-routed vs. direct-to-service

Architecture Overview

The system consists of 10 Docker containers: 2 Blazor frontends, 1 API Gateway (YARP), 6 microservices (each with its own SQLite database), and RabbitMQ for async messaging. SimpleFrontend calls services through the API Gateway; WebFrontend calls them directly.



Complete Service Map (10 containers)

- API Gateway (YARP) :5000 — reverse proxy routing + aggregated endpoint
- CustomerService :5001 — buyers/sellers management, role-based dashboards
- OrderService :5002 — order creation, status transitions, RabbitMQ publisher
- ProductService :5003 — product catalog, stock, discounts
- CartService :5005 — shopping cart with checkout orchestration
- PaymentService :5006 — payment processing (mock 90% approval)
- NotificationService :5007 — notifications + RabbitMQ consumers
- RabbitMQ :5672/:15672 — message broker (management UI on 15672)
- SimpleFrontend :5010 — simple Blazor dashboard with data tables
- WebFrontend :5020 — AI-generated FDU-themed e-commerce marketplace

SimpleFrontend (port 5010)

A minimal Blazor Server frontend using the default project template with Bootstrap tables. It calls the API Gateway and displays raw data from every microservice. No custom styling — it uses the default Blazor template with standard Bootstrap classes. The goal is to demonstrate the frontend consuming all backend endpoints via IHttpClientFactory.

Project Structure

The project is a .NET 10 Blazor Server app. Program.cs registers IHttpClientFactory with a named 'Api' client pointing to the API Gateway. The NavMenu has 7 links: Home, Customers, Products, Orders, Cart, Payments, and Notifications. The Dockerfile uses a multi-stage build with sdk:10.0-alpine and aspnet:10.0-alpine images.

Pages

- Home (/) — dashboard showing service health with OK/Error badges and record counts
- Customers (/customers) — auto-loaded table with ID, Name, Email, Phone, Role badge, Address, Active, Created
- Products (/products) — auto-loaded table with ID, Seller, Name, Category, Price, Stock, Discount%, Final Price
- Orders (/orders) — order table plus Create Order form (Buyer ID, Product ID, Qty) that triggers RabbitMQ event
- Cart (/cart) — lookup by Buyer ID with cart items table and Add Item form
- Payments (/payments) — auto-loaded table with ID, Order, Buyer, Amount, Method, Status badge, timestamps
- Notifications (/notifications) — lookup by User ID showing RabbitMQ-generated entries

Key Technical Details

- Uses IHttpClientFactory with named client 'Api' — base URL set via Services__ApiGateway env var
- All pages use @rendermode InteractiveServer for real-time interactivity
- Create Order form on Orders page publishes OrderCreatedEvent via RabbitMQ through the backend
- Docker: runs on port 5010, depends_on apigateway

WebFrontend (port 5020)

A fully styled Blazor e-commerce frontend built with AI-assisted web scraping and code generation. The concept is a course-selling platform where FDU sellers list courses/products and buyers can browse, add to cart, place orders, and receive notifications.

How It Was Built

The FDU website (fdu.edu) was scraped to capture the university's visual identity — colors, typography, layout patterns, and overall design language. Using the scraped reference material, GitHub Copilot generated all the custom CSS, HTML structure, component layouts, and responsive design from scratch. No FDU assets, images, or copyrighted content were copied; only the visual style was used as inspiration.

Project Structure

The project is a .NET 10 Blazor Server app. Program.cs registers 6 typed HttpClient services (one per microservice, direct calls) plus a ProfileState for role switching. The Models folder has Customer, Product, Order, Cart, Payment, and Notification classes. The Services folder has a typed service class per microservice plus ProfileState. The wwwroot/app.css contains FDU-inspired custom CSS with maroon/burgundy primary, gold accents, and dark tones.

Pages

- Home (/) — landing page with FDU-themed hero section and marketplace overview
- Dashboard (/dashboard) — role-based dashboard (Admin/Buyer/Seller) with stats and quick actions
- Customers (/customers) — customer management with role badges and detail views
- Products (/products) — product catalog with seller info, discounts, stock management
- Orders (/orders) — order management with status transitions and cancel/return flows
- Cart (/cart) — shopping cart with checkout orchestration
- Payments (/payments) — payment history with status badges and processing
- Notifications (/notifications) — notification center with read/unread management

Key Features

- FDU-inspired color scheme: CSS variables with maroon/burgundy primary, gold accents, dark backgrounds
- Role-based ProfileState: Admin sees everything, Buyer sees own orders/cart, Seller sees own products/sales
- Typed HttpClient services per microservice — no API Gateway dependency, calls services directly
- Full CRUD operations on all pages with forms and validation
- Responsive layout with Blazor CSS isolation (.razor.css files)
- Docker: runs on port 5020, env vars configure direct service URLs

API Communication Patterns

The two frontends demonstrate two different approaches to calling backend services.

SimpleFrontend: Via API Gateway

SimpleFrontend uses a single named HttpClient ('Api') pointing to the API Gateway on port 5000. All requests go through YARP, which routes them to the appropriate microservice. This is the typical production pattern — clients know only the gateway address.

WebFrontend: Direct to Services

WebFrontend uses 6 typed HttpClient services, each configured with the direct URL of a microservice (e.g. http://customerservice:8080). This bypasses the API Gateway entirely. Service URLs are injected via environment variables in docker-compose.yml. This pattern demonstrates service-to-service communication without a gateway.

Docker Compose Updates

Two new services were added to docker-compose.yml, bringing the total to 10 containers.

- simplefrontend — builds from SimpleFrontend/SimpleFrontend, port 5010->8080, env var Services__ApiGateway=http://apigateway:8080, depends_on apigateway
- webfrontend — builds from WebFrontend/WebFrontend, port 5020->8080, env vars for 6 direct service URLs, depends_on apigateway

Other changes: .gitignore was updated to include the WebFrontend directory, DistributedDataAccessLab.sln was updated to include SimpleFrontend, and README.md was updated with the full 10-service table and a V3 Plus section explaining the AI workflow.

Frontend Comparison

The two frontends serve different purposes and demonstrate different approaches.

- Styling: SimpleFrontend uses the default Blazor template with Bootstrap; WebFrontend uses custom AI-generated FDU-themed CSS
- API calls: SimpleFrontend routes through the API Gateway; WebFrontend calls each microservice directly
- Pages: SimpleFrontend has 7 pages with data tables; WebFrontend has 8+ pages with full CRUD forms
- Role-based UI: SimpleFrontend has none; WebFrontend has Admin/Buyer/Seller profile switching
- CSS: SimpleFrontend uses standard Bootstrap; WebFrontend uses CSS variables and Blazor CSS isolation
- Purpose: SimpleFrontend is a simple functional demo; WebFrontend is a production-like AI-generated marketplace

AI as a Development Accelerator

The WebFrontend demonstrates how AI can be used as a development tool. The entire workflow was: (1) scrape the FDU website for design reference, (2) feed the visual patterns to GitHub Copilot, (3) generate all CSS variables, component layouts, and responsive breakpoints, (4) integrate with the microservices backend. No copyrighted assets were used — only the visual style as inspiration.

Conclusion

This deliverable adds two Blazor Server frontends to the microservices backend. SimpleFrontend provides a minimal dashboard consuming all APIs through the gateway, while WebFrontend showcases a fully styled AI-generated marketplace with role-based UI and direct service communication. The full system now runs 10 Docker containers with a single docker compose up --build command.